

Shiv Nadar University Chennai

B.Tech Artificial Intelligence and Data Science

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Name: Prithviraj RP

Register Number: 24011101079

Semester: V

Academic Year: 2026–27

GitHub: <https://github.com/Prithvi1247/Deep-Learning-Lab->

Contents

1	Objective	2
2	Learning Outcomes	2
3	Background Theory	2
4	Dataset and Experimental Setup	5
5	Experimental Procedure	5
6	Experimental Results	8
7	Hyperparameter Analysis	9
8	Transfer Learning and Fine-Tuning	10
9	Five-Fold Cross-Validation	10
10	Final Model Evaluation	11
11	Class-wise Evaluation	11
12	Mandatory Plots	13
13	Overall Results	24
14	Discussion	25
15	Additional Exercise	28
16	Conclusion	29
17	References	30

1 Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Individual design choices are studied using training and validation curves, followed by five-fold cross-validation for model selection. The final selected configuration is then retrained on the complete training data and evaluated on the independent test set.

2 Learning Outcomes

After completing this experiment, the following concepts were studied:

Different weight initialization strategies.

Identification of overfitting using training and validation curves.

Comparison of SGD, Momentum, RMSProp and Adam.

CNN hyperparameters such as kernel size, stride, padding and pooling.

Batch Normalization and Dropout.

Transfer learning and fine-tuning using MobileNetV2.

Learning-rate, batch-size and dropout tuning.

Five-fold cross-validation for model selection.

Model selection using accuracy, variability and computational cost.

3 Background Theory

MobileNetV2

MobileNetV2 is a lightweight convolutional neural network designed to obtain useful image representations with comparatively low computational cost. Its important components include depthwise convolution, pointwise 1×1 convolution, inverted residual blocks, linear bottlenecks, Batch Normalization and ReLU6 activation.

For this experiment, an ImageNet-pretrained MobileNetV2 was used with its original classification head removed. A Global Average Pooling layer and a new classifier were then added for the 37 Oxford-IIIT Pet classes.

Weight Initialization

Weight initialization determines the initial values of trainable parameters before optimization. The experiment compares Zero, Random, Xavier/Glorot and He initialization. A suitable initialization can influence convergence speed and stability.

Regularization

Regularization methods reduce overfitting by constraining the model or its activations. The experiment compares no regularization, L2 regularization, Dropout and Batch Normalization.

For L2 regularization,

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_i w_i^2.$$

Dropout randomly disables a fraction of activations during training, reducing excessive dependence on individual neurons.

Batch Normalization

For a mini-batch $x_1, \dots, x_m$, the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

and the output is

$$y_i = \gamma \hat{x}_i + \beta.$$

For the numerical example $x = [2, 4, 6, 8]$,

$$\mu_B = 5, \quad \sigma_B^2 = 5,$$

giving

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342].$$

When $\gamma = 1$ and $\beta = 0$, these normalized values are also the final outputs.

Optimization

The optimizer controls how gradients update model parameters. SGD uses the current gradient directly, Momentum incorporates a velocity term based on previous updates, RMSProp adapts the learning rate using a moving average of squared gradients, and Adam combines adaptive first- and second-moment estimates.

Transfer Learning

Transfer learning reuses representations learned from a large source dataset. In this experiment, ImageNet-pretrained MobileNetV2 provides the feature extractor.

In feature extraction, the pretrained base remains frozen and only the new classifier is trained. In fine-tuning, selected upper layers of the pretrained base are unfrozen and trained using a smaller learning rate.

K-Fold Cross-Validation

For five-fold cross-validation, the training data is divided into five folds. Each fold is used once as validation data while the remaining four folds are used for training.

The mean accuracy is

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and the standard deviation is

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

The final result is reported as $\bar{A} \pm SD$. The independent test set is not used during model selection.

4 Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

Classes: 37 cat and dog breeds

Input Size: $224 \times 224 \times 3$

Model: MobileNetV2 pretrained on ImageNet

Task: Multi-class image classification

All RGB images were resized to $224 \times 224 \times 3$ and normalized according to the pretrained MobileNetV2 requirements. Training, validation and test data were kept separate, and the independent test set was reserved for final evaluation.

The final selected model contained 2,305,381 total parameters, of which 47,397 were trainable.

5 Experimental Procedure

Task 1: Weight Initialization

Four initialization strategies were investigated:

Zero

Random

Xavier/Glorot

He

The best validation accuracy was obtained with Zero initialization at approximately 91.30%.

Task 2: Regularization and Overfitting

The following configurations were compared:

No Regularization

L2

Dropout

Batch Normalization

The comparison was performed using training and validation loss and accuracy curves. The no-regularization configuration produced the highest validation accuracy among the tested regularization configurations, while Dropout reduced the training accuracy and produced a smaller training–validation gap.

Task 3: Batch Normalization

A numerical Batch Normalization example was implemented using the input vector

$$[2, 4, 6, 8].$$

The resulting output was

$$[-1.34163944, -0.44721315, 0.44721315, 1.34163944].$$

A direct comparison of models with and without Batch Normalization was also performed.

Task 4: Optimizer Comparison

SGD, Momentum, RMSProp and Adam were compared. RMSProp provided the strongest validation performance in the optimizer experiment, reaching approximately 91.17% validation accuracy.

Task 5: Hyperparameter Tuning

The controlled hyperparameter experiments varied one parameter at a time.

Hyperparameter	Tested Values	Best Observed Value
Learning Rate	0.001, 0.0001	0.001
Batch Size	16, 32, 64	32 / 64 tied on best CV-style metric
Dropout	0, 0.25, 0.5	0

For the learning-rate experiment, 0.001 achieved 91.44% best validation accuracy, whereas 0.0001 achieved 83.70%.

For dropout, the rate of 0 achieved the highest validation accuracy of 91.98%, while dropout rates of 0.25 and 0.5 achieved lower validation accuracy.

Task 6: Transfer Learning and Fine-Tuning

Feature extraction and fine-tuning were compared. Feature extraction achieved the better result:

90.90%

best validation accuracy, compared with approximately 90.22% for fine-tuning.

Fine-tuning used a learning rate of 10^{-4} and increased the number of trainable parameters to approximately 1.25 million. It produced a much lower training loss but a larger generalization gap.

Task 7: Five-Fold Cross-Validation

Three promising configurations were selected:

C1 – Baseline

C2 – Zero Initialization

C3 – Fine-Tuning

Each configuration was evaluated using five-fold cross-validation on the training data. The independent test set was not used during this selection stage.

6 Experimental Results

Weight Initialization Results

Initialization	Best Val. Accuracy (%)	Final Train Loss	Time (s)
Zero	91.3044	0.090472	75.65
Random	90.6250	0.105421	73.69
Xavier	90.4891	0.099241	70.82
He	90.8967	0.098784	71.24

Inference: Zero initialization had the greatest validation accuracy when using this particular transfer-learning classifier. The four initialization approaches had similar results but differed in how they learned.

Regularization Results

Method	Best Val. Acc. (%)	Final Train Acc. (%)	Final Val. Acc. (%)	Time (s)
No Regularization	91.0326	98.7092	91.0326	99.29
L2	90.7609	98.8791	90.7609	109.75
Dropout	90.6250	93.5462	90.0815	107.40
Batch Normalization	90.6250	99.5245	90.4891	88.91

Inference: Validation accuracy was the best for the no-regularization model among the other configurations. Dropout had a significant impact on decreasing the accuracy in training, but it failed to increase the validation accuracy.

Batch Normalization Comparison

Configuration	Best Val. Acc. (%)	Final Val. Acc. (%)	Time (s)
Without BN	90.7609	90.4891	94.24
With BN	90.2174	90.2174	80.68

Inference: In this controlled comparison, the model without Batch Normalization had better validation accuracy. However, the model with Batch Normalization converged with a lower loss in training and less training time.

Optimizer Results

Comparison between the four optimizers was made using validation-accuracy graphs. It was evident that there was significantly slower learning with SGD while the Momentum optimizer progressively improved. The RMSProp and Adam optimizers attained high validation accuracy very fast. The highest validation accuracy recorded by the RMSProp algorithm was about 91.17%.

Inference: Adaptive optimization was more effective than SGD optimization in this experiment. The RMSProp optimizer attained high validation accuracy very fast.

7 Hyperparameter Analysis

Learning Rate

Learning Rate	Best Val. Acc. (%)	Final Val. Acc. (%)	Time (s)
0.001	91.4402	90.3533	81.54
0.0001	83.6957	83.6957	87.06

Inference: The result from a learning rate of 0.001 optimized much better and faster than that with 0.0001. The latter was far too small to produce similar results after five passes.

Batch Size

Batch Size	Best Val. Acc. (%)	Final Val. Acc. (%)	Time (s)
16	90.0815	89.9457	99.94
32	90.7609	90.6250	70.46
64	90.7609	90.2174	89.71

Inference: The batch size that produced the optimal combination of model performance and time to train was 32. Using 16 for the batch size meant training took more time, whereas using 64 for the batch size did not boost validation accuracy.

Dropout Rate

Dropout	Best Val. Acc. (%)	Final Val. Acc. (%)	Time (s)
0	91.9837	91.9837	87.52
0.25	90.8967	90.8967	78.17
0.5	90.7609	90.7609	87.13

Inference: Without the use of dropout, the validation accuracy was highest. The more the dropout rate increased, the lower the training accuracy and no improvement was seen during validation.

8 Transfer Learning and Fine-Tuning

Method	Learning Rate	Trainable Parameters	Best Val. Acc. (%)	Time (s)
Feature Extraction	0.001	47,397	90.90	88.26
Fine-Tuning	0.0001	1,253,477	90.22	85.27

The fine-tuning process had a training loss of 0.0348 and validation loss of 0.3275. However, feature extraction offered better validation results using fewer parameters to train.

Inference: Finetuning helped the pre-trained representation to become more adaptable to the training set, thereby giving a much smaller training loss. Nevertheless, the validation set accuracy reduced, which means that the added trainable layers were not generalizing well.

9 Five-Fold Cross-Validation

The three selected configurations were evaluated using five-fold cross-validation.

Configuration	Mean Accuracy (%)	SD (%)	Mean Time (s)
C1 – Baseline	91.5461	0.3411	100.66
C2 – Zero Initialization	91.7105	0.7410	106.37
C3 – Fine-Tuning	90.1974	0.8979	101.54

The five-fold results were:

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1 – Baseline	91.78	91.61	91.94	91.12	91.28	91.55 $\pm$ 0.34
C2 – Zero Init.	92.76	91.28	91.94	90.79	91.78	91.71 $\pm$ 0.74
C3 – Fine-Tuning	90.95	90.46	90.30	88.65	90.63	90.20 $\pm$ 0.90

Inference: The best mean cross-validation accuracy obtained through zero initialization was 91.71%. The baseline performed better with an average of 91.55%, though it had a smaller standard deviation of 0.34%. Fine tuning gave the worst mean accuracy with higher variance.

10 Final Model Evaluation

The final configuration selected using five-fold cross-validation was:

Initialization: Zero

Optimizer: RMSProp

Learning Rate: 0.001

Batch Size: 32

Dropout: 0

Total Parameters: 2,305,381

Trainable Parameters: 47,397

The selected configuration was retrained using the complete training data. Only after this stage was the independent test set evaluated.

Metric	Value
Mean CV Accuracy	91.71%
CV Standard Deviation	0.74%
Test Accuracy	89.97%
Macro Precision	90.54%
Macro Recall	89.91%
Macro F1-score	89.93%
Training Time	84.32 s
Total Parameters	2,305,381
Trainable Parameters	47,397
Final Training Accuracy	98.37%
Final Training Loss	0.0767
Test Loss	0.3092

Inference: The last model produced an accuracy of 89.97% with a macro-F1 score of 89.93%. The gap of 91.71% on cross-validation and 89.97% on testing suggests a moderate generalization gap, while the high accuracy of 98.37% for training signifies a good fit to the training data.

11 Class-wise Evaluation

Class	Precision	Recall	F1	Support
Abyssinian	0.88	0.86	0.87	98
american bulldog	0.66	0.93	0.78	100
american pit bull terrier	0.83	0.62	0.71	100
basset hound	0.94	0.92	0.93	100
beagle	0.92	0.91	0.91	100
Bengal	0.71	0.88	0.79	100
Birman	0.83	0.77	0.80	100
Bombay	0.90	0.89	0.89	88
boxer	0.93	0.79	0.85	99
British Shorthair	0.93	0.65	0.76	100
chihuahua	0.94	0.91	0.92	100
Egyptian Mau	0.94	0.77	0.85	97
english cocker spaniel	0.99	0.91	0.95	100
english setter	0.93	0.96	0.95	100
german shorthaired	0.95	0.99	0.97	100
great pyrenees	1.00	0.97	0.98	100
havanese	0.92	0.99	0.95	100
japanese chin	0.99	1.00	1.00	100
keeshond	0.97	1.00	0.99	99
leonberger	0.99	0.98	0.98	100
Maine Coon	0.82	0.87	0.84	100
miniature pinscher	0.97	0.89	0.93	100
newfoundland	0.99	0.99	0.99	100
Persian	0.89	0.90	0.90	100
pomeranian	0.99	0.89	0.94	100
pug	0.99	0.94	0.96	100
Ragdoll	0.74	0.75	0.74	100
Russian Blue	0.71	0.89	0.79	100
saint bernard	0.95	0.97	0.96	100
samoyed	0.93	0.99	0.96	100
scottish terrier	0.97	1.00	0.99	99
shiba inu	0.94	0.97	0.96	100
Siamese	0.92	0.88	0.90	100
Sphynx	0.93	0.96	0.95	100
staffordshire bull terrier	0.67	0.74	0.71	89
wheaten terrier	0.97	0.94	0.95	100
yorkshire terrier	0.98	1.00	0.99	100

Japanese Chin performed exceptionally well with respect to classification with F1 score of 1.00. The other breeds that performed exceptionally well in terms of classification were Keeshond, Newfoundland, Scottish Terrier, and Yorkshire Terrier. On the other hand, breeds such as American Pit Bull Terrier and Staffordshire Bull Terrier had poor F1 scores.

12 Mandatory Plots

9.1 Training Loss for Different Weight Initializations

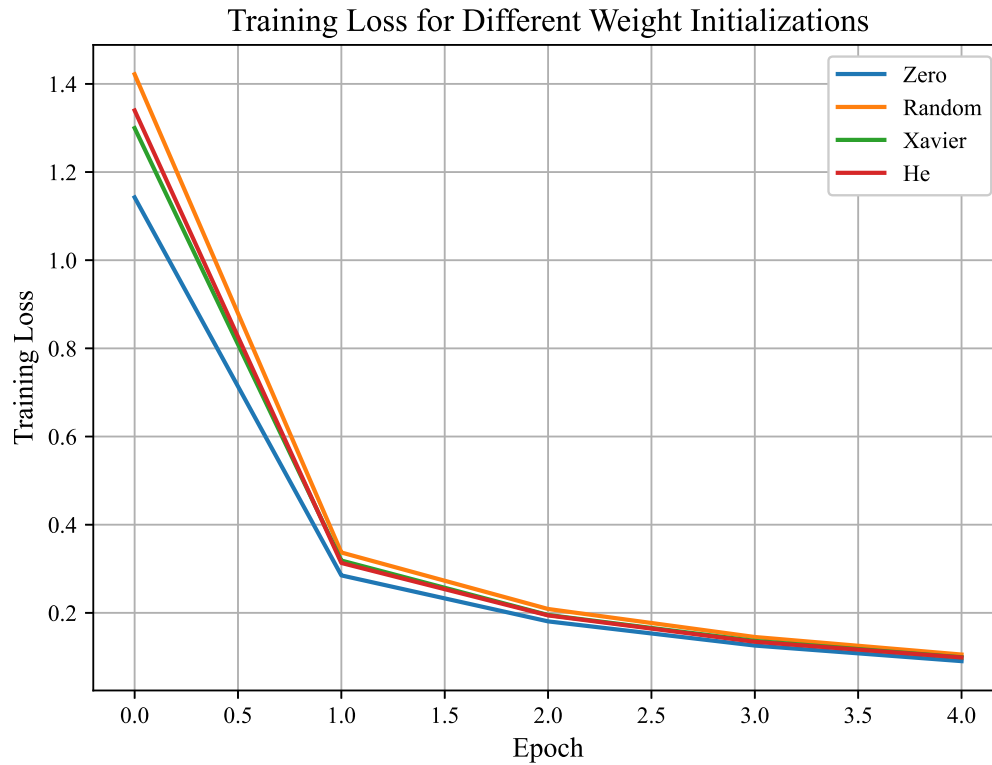


Figure 1: Training Loss for Different Weight Initializations

Inference: Training loss drops significantly in the first few epochs for all the initialization techniques. Afterwards, they converge to similar low levels, indicating that the impact of different initialization schemes is primarily at the start of training.

9.2 Validation Accuracy for Different Weight Initializations

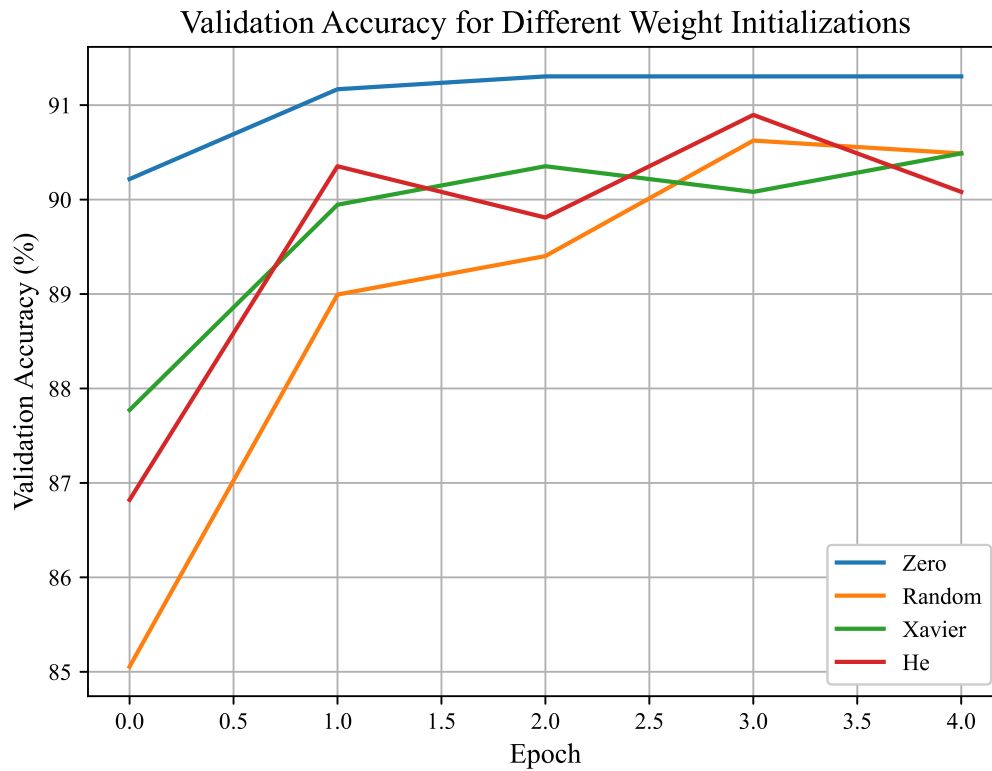


Figure 2: Validation Accuracy for Different Weight Initializations

Inference: Zero initialization shows the highest value of accuracy among all methods. Random, Xavier, and He initialization demonstrate almost identical results of about 90–91% of convergence but with different learning curves.

9.3 Training and Validation Accuracy for Regularization

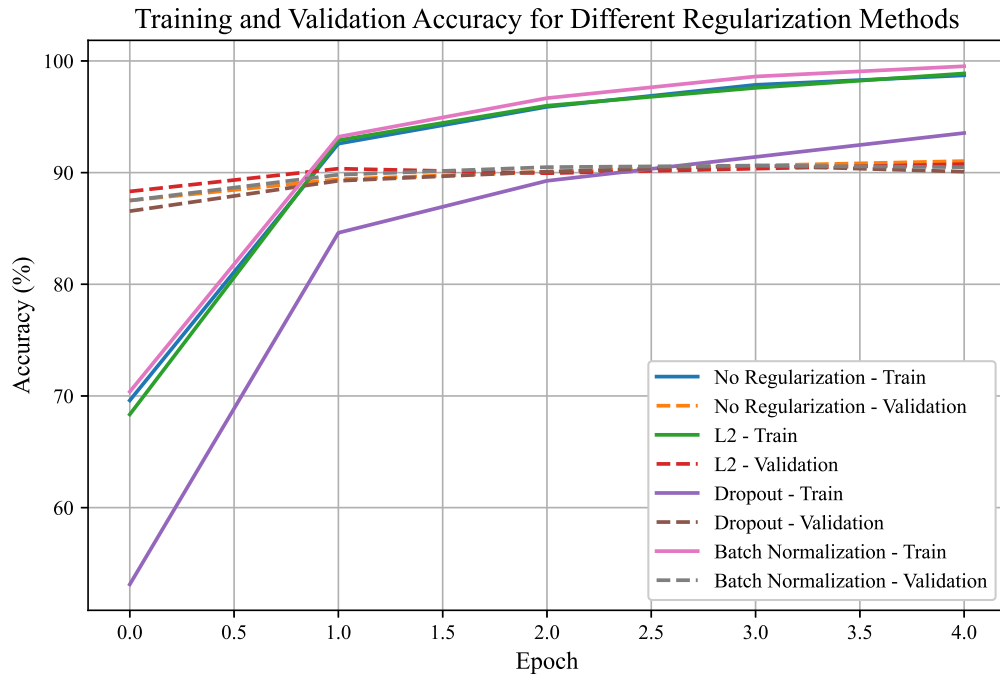


Figure 3: Training and Validation Accuracy for Different Regularization Methods

Inference: Training accuracy levels for the methods start to differ as training continues, but validation accuracy remains at 90–91%. Dropout method gives the lowest training accuracy level, showing its regularization ability.

9.4 Training and Validation Loss for Regularization

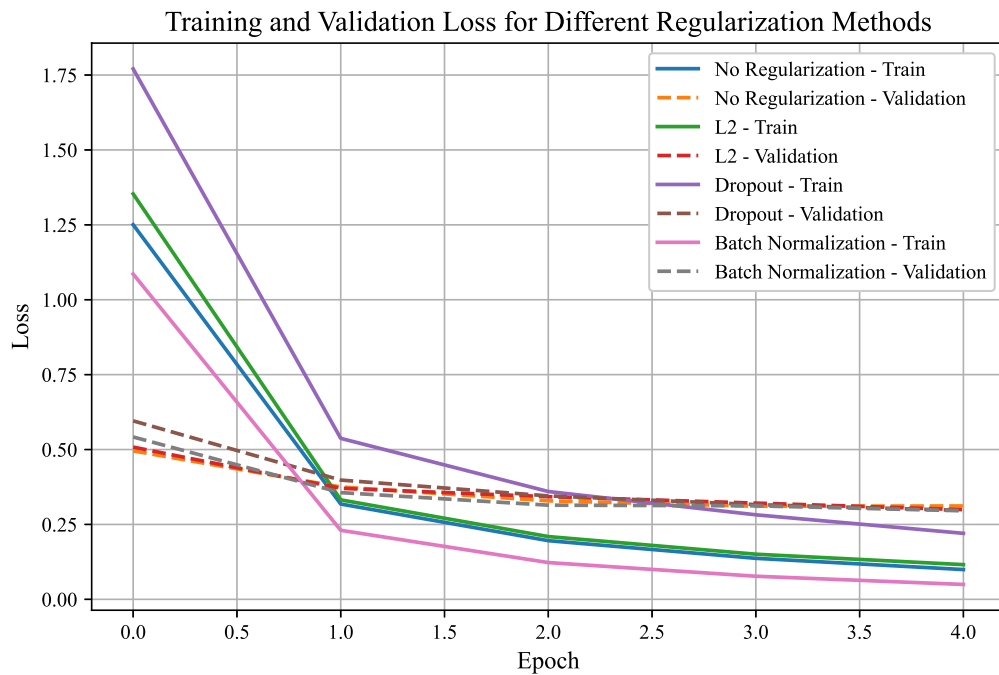


Figure 4: Training and Validation Loss for Different Regularization Methods

Inference: The training loss decreases continuously while the validation loss tends to remain constant at high values. This phenomenon becomes clearer in the case of models which have high correlation with the training dataset.

9.5 Batch Normalization Comparison

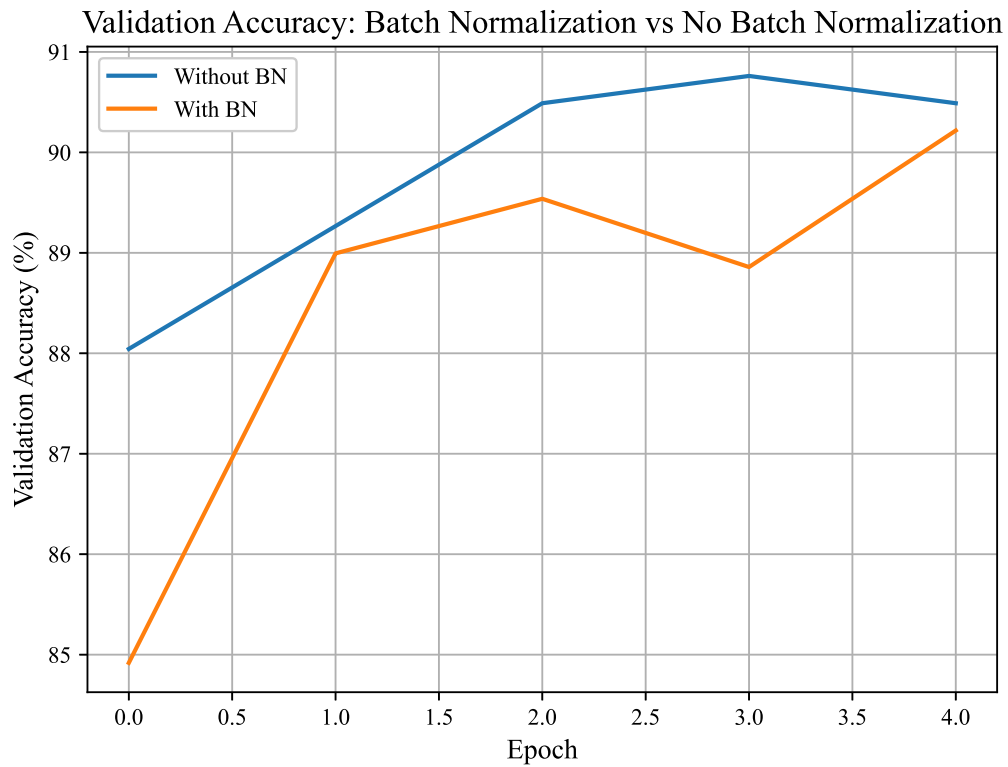


Figure 5: Validation Accuracy: Batch Normalization vs. No Batch Normalization

Inference: The model without batch normalization has better validation accuracy in this experiment. However, batch normalization helps to decrease the training loss faster, revealing its influence on optimization.

9.6 Validation Accuracy for Different Optimizers

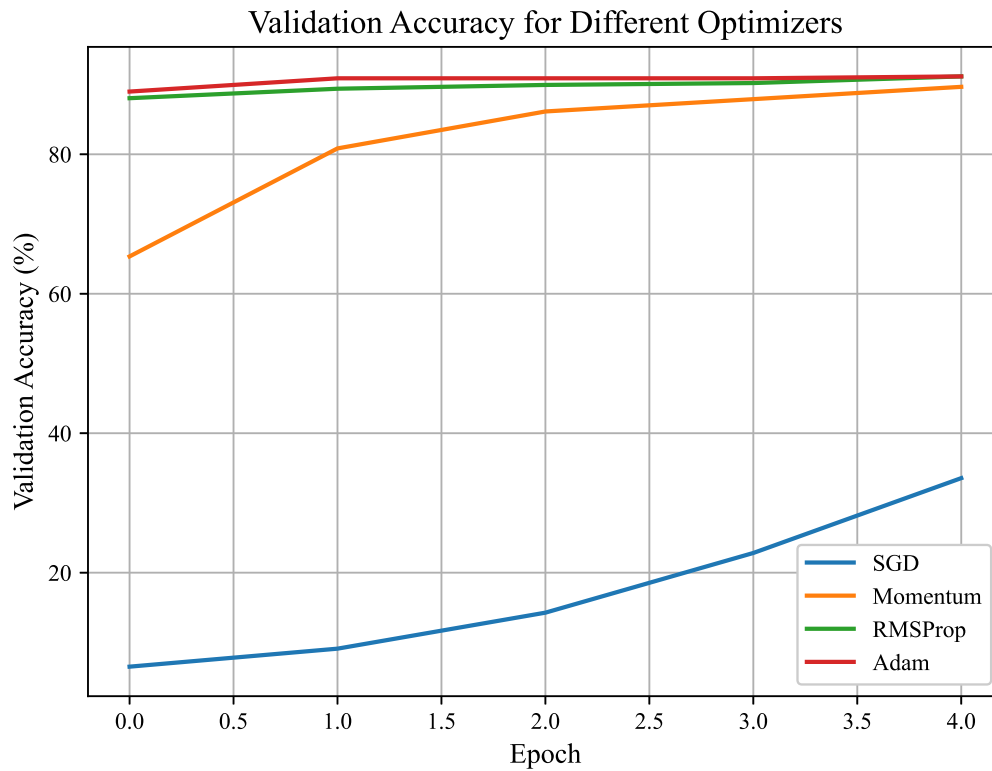


Figure 6: Validation Accuracy for Different Optimizers

Inference: SGD is significantly slower compared to the adaptive methods. Both RMSProp and Adam achieve high validation accuracy quickly, with RMSProp delivering the best performance among all models.

9.7 Learning Rate Comparison

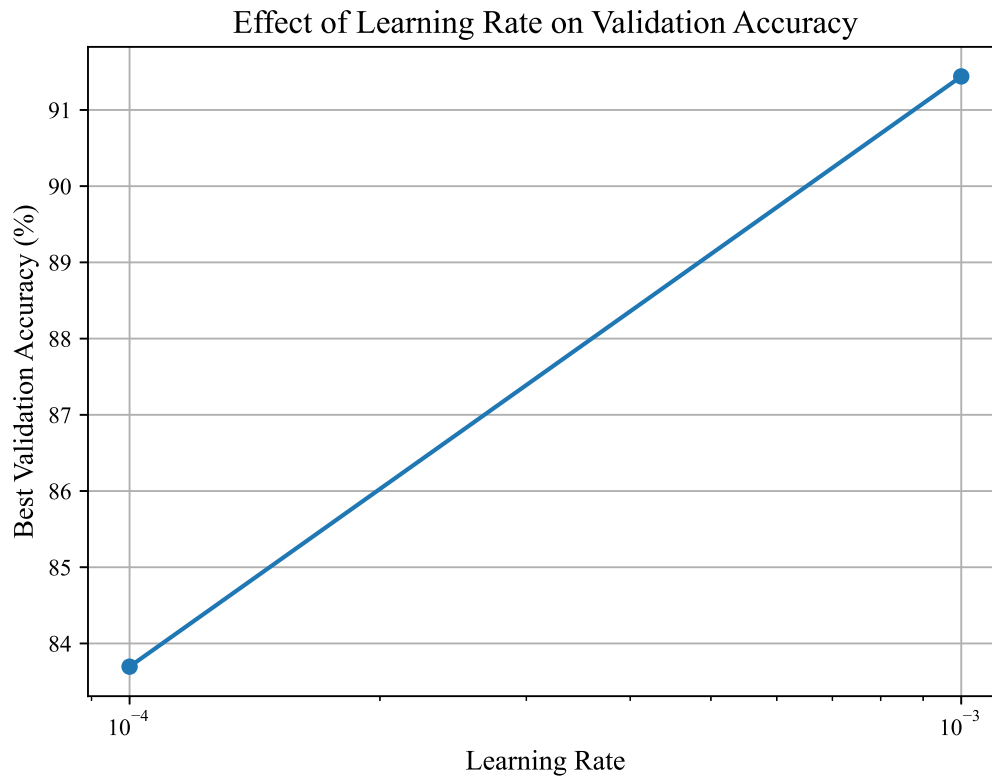


Figure 7: Validation Accuracy for Different Learning Rates

Inference: A learning rate of 0.001 is significantly more effective than 0.0001 given the five-epoch time limit. The lower learning rate causes slow convergence and much lower validation accuracy.

9.8 Batch Size Comparison

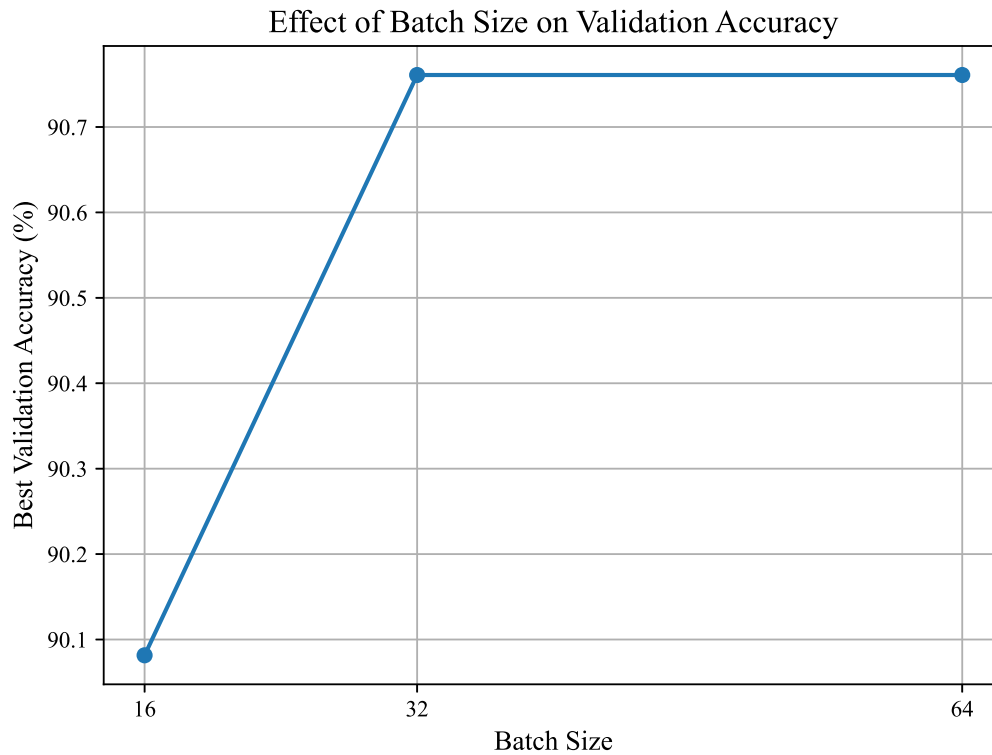


Figure 8: Validation Accuracy for Different Batch Sizes

Inference: The batch sizes of 32 and 64 have comparable best accuracies for the validation data. The batch size of 16 yields lower results. However, the optimal value for this metric is 32.

9.9 Dropout Rate Comparison

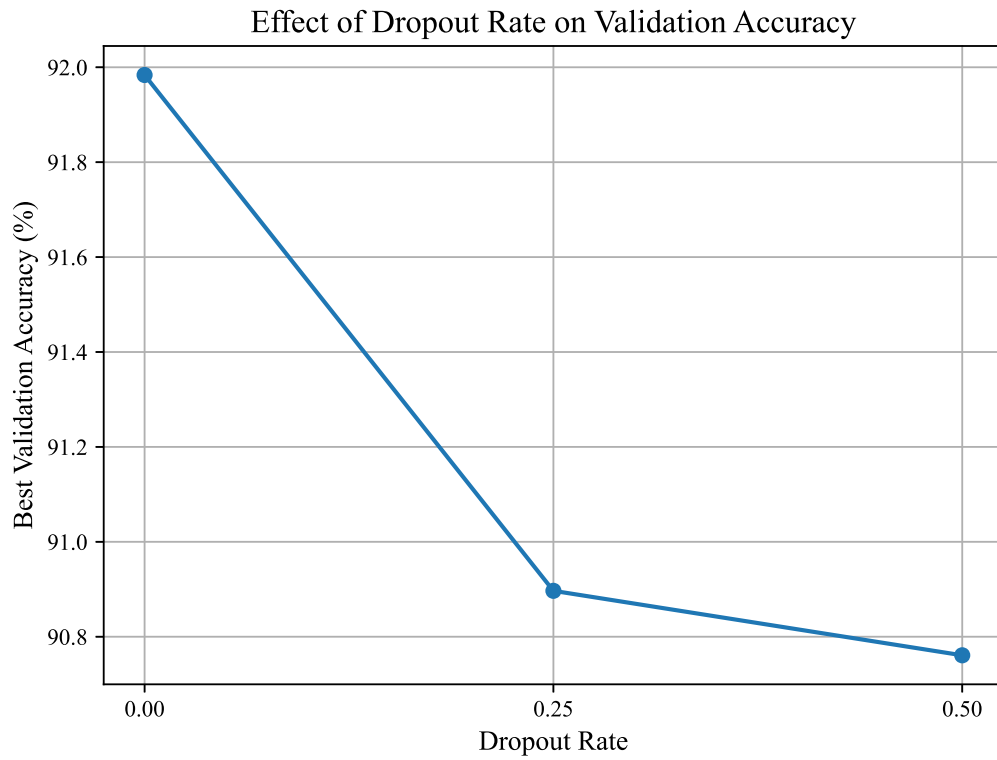


Figure 9: Validation Accuracy for Different Dropout Rates

Inference: The setup without dropout produces the best validation accuracy. As the dropout ratio increases, the validation accuracy drops for the particular setup, suggesting that further regularization was not needed.

9.10 Training and Validation Loss: Transfer Learning

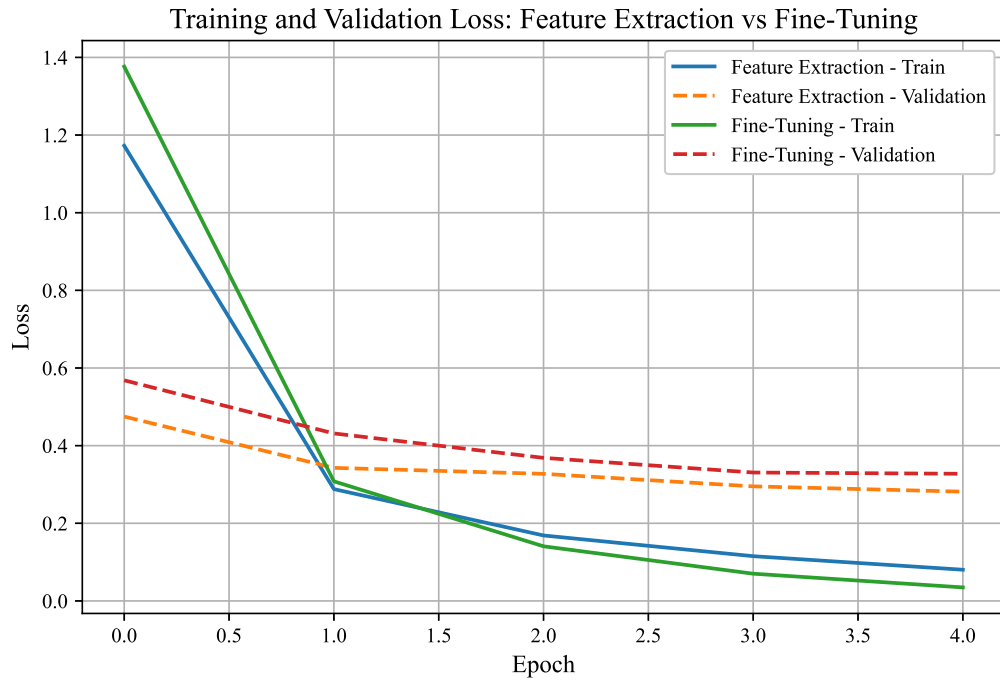


Figure 10: Training and Validation Loss for Feature Extraction and Fine-Tuning

Inference: Fine tuning has a smaller loss on the training data, reflecting better fitting to the training data. Validation loss is larger for fine-tuned data, reflecting the fact that there is no improvement in generalization due to fine tuning.

9.11 Five-Fold Cross-Validation

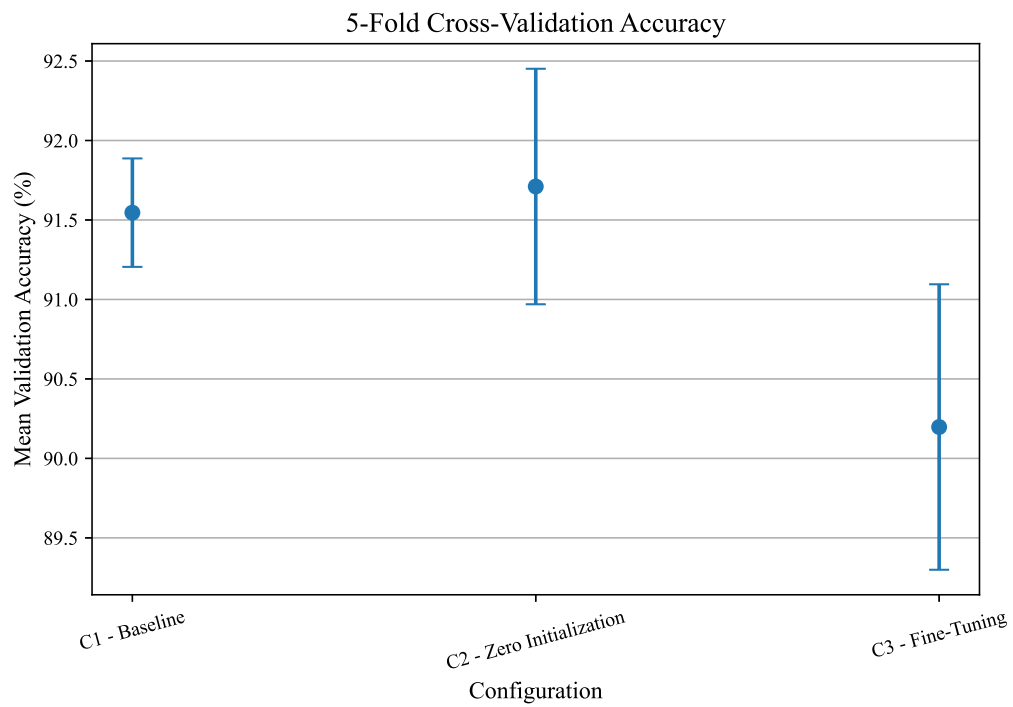


Figure 11: 5-Fold Cross-Validation Accuracy with Standard-Deviation Error Bars

Inference: "Zero Initialization" has the greatest accuracy rate for cross validation at 91.71%. "Baseline" is less prone to variance since its standard deviation is lower, whereas "Fine-Tuning" has the smallest mean and largest variation.

9.12 Final Confusion Matrix

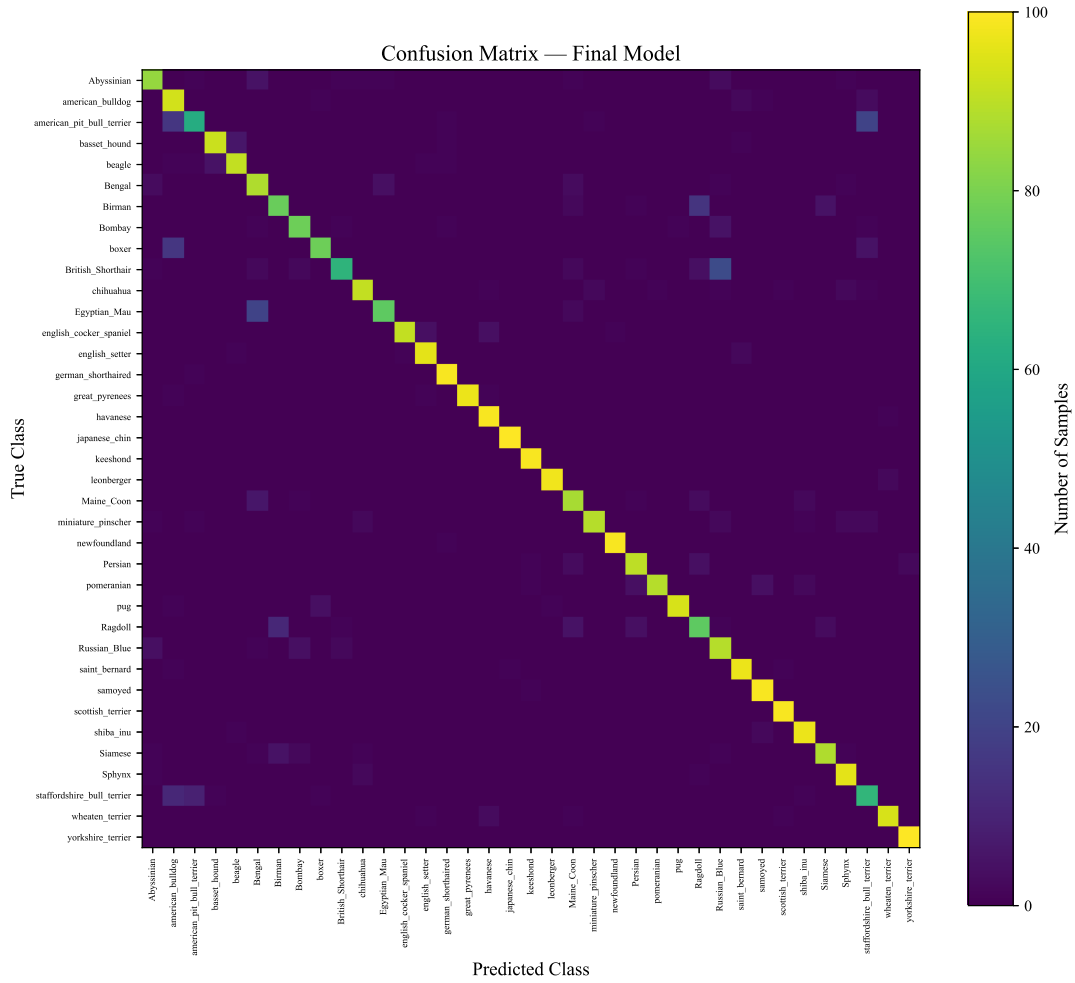


Figure 12: Confusion Matrix for the Final Model

Inference: The confusion matrix is quite diagonal and indicates that the majority of pet breeds have been correctly classified. The off-diagonal elements that are noticeable are found between breeds that share similar visual characteristics, such as the American Pit Bull Terrier, Staffordshire Bull Terrier and related breeds.

13 Overall Results

Stage	Selected Configuration	Validation/CV (%)	Test (%)	Time
Baseline	RMSProp, LR 0.001, Batch 32, Dropout 0	91.03	—	99.29 s
Best Initialization	Zero	91.30	—	75.65 s
Best Regularization	No Regularization	91.03	—	99.29 s
Best Optimizer	RMSProp	91.17	—	—
Best Hyperparameters	LR 0.001, Batch 32, Dropout 0	91.98	—	87.52 s

Feature Extraction	Frozen MobileNetV2	90.90	–	88.26 s
Fine-Tuned Model	LR 10^{-4} , partial unfreezing	90.22	–	85.27 s
Final Model	Zero + RMSProp + LR 0.001 + Batch 32 + Dropout 0	91.71 CV	89.97	84.32 s

Final selection: The final model was selected using five-fold cross-validation rather than the highest single validation score. Zero initialization achieved the highest mean CV accuracy of 91.71%, with a standard deviation of 0.74%. The final independent test accuracy was 89.97%, with macro precision of 90.54%, macro recall of 89.91% and macro F1-score of 89.93%.

14 Discussion

1. What is the difference between model parameters and hyperparameters?

Parameters of the model are those which are learned during the training process, e.g., convolution kernels, biases. Hyperparameters are those which are selected prior to training, e.g., learning rate, batch size, dropout, optimizer.

2. Why is weight initialization important?

The initialization step defines the initial position in the optimization process. Unfavorable initial positions may slow down the optimization process or cause unstable activations, whereas appropriate initialization can enhance the optimization process.

3. Why can zero initialization be problematic for neural networks?

When all neurons in a trainable layer have an identical initialization, they can also get identical gradient values and learn the same representation. As such, symmetry will not enable differentiation of different neurons. In this case, although the newly added classifier used zero initialization, this still resulted in the best validation value achieved.

4. Compare Xavier and He initialization.

While Xavier initialization works well for symmetrical activation functions like the tanh function, He initialization is specifically formulated keeping in mind the variance needed for the

ReLU family of activation functions.

5. How can training and validation curves identify overfitting?

If there is an improvement in the training performance but no improvement in validation performance, then overfitting takes place. The gap between training and validation indicates that the model fits the training data better than the unseen data.

6. How does Dropout reduce overfitting?

Dropout works by randomly dropping some activations when training, which prevents the model from being too dependent on certain neurons.

7. What is the purpose of Batch Normalization?

In Batch Normalization, normalization is performed by making use of statistics from a mini-batch followed by learnable scaling and shifting factors.

8. Explain the numerical Batch Normalization example.

For $x = [2, 4, 6, 8]$, the mean is 5 and the variance is 5. Dividing the deviations from the mean by $\sqrt{5}$ produces approximately $[-1.342, -0.447, 0.447, 1.342]$.

9. What are the roles of γ and β ?

γ controls the learned scale of the normalized activation and β controls the learned shift.

10. Compare SGD, Momentum, RMSProp and Adam.

In SGD, the current gradient is used to update parameters. Momentum takes into account information from past gradients. RMSProp adjusts the learning rate based on an exponential moving average of the square of the gradient. Adam computes adaptive estimates of first- and second-order moments..

11. What happens when the learning rate is too large?

These updates may be too large, making the process unstable, leading to oscillations around the minima or even divergence.

12. What happens when the learning rate is too small?

It makes the optimization process too slow to converge to a good solution in the limited training budget.

13. What is the effect of increasing batch size?

More training data implies better gradient estimation stability, yet fewer weight updates within one epoch. It can have implications for memory consumption and training duration as well.

14. Explain stride and padding.

Stride denotes the distance the filter travels from one point to another. Padding increases the number of pixels on the edges of the input image.

15. Why is MobileNetV2 computationally efficient?

MobileNetV2 uses depthwise separable convolutions, inverted residual blocks and linear bottlenecks to reduce computation and parameter requirements.

16. What is depthwise separable convolution?

The technique decomposes the convolution process into a depthwise convolution that is performed individually on each input channel and then a pointwise convolution of size 1×1 . It involves less computation than the traditional convolution.

17. What is transfer learning?

Transfer learning uses knowledge learned by a model on a large source dataset and adapts it to a different target task.

18. Differentiate feature extraction and fine-tuning.

Feature extraction involves locking the pretrained base model and training only the newly added classifier layer. Fine-tuning involves unlocking certain pretrained layers and fine-tuning them for the target task.

19. Why is a smaller learning rate used during fine-tuning?

The pre-trained layers already have valuable representations. With a larger learning rate, the representations might be updated too drastically, hence the need for a smaller learning rate.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

It assesses the configuration across various train-validate splits and does not depend on a particular split only. This makes the assessment of average performance more accurate and valid.

21. Why must the test set remain untouched during tuning?

The use of the test set in model selection results in information leakage. The test set should be used for independent final testing purposes only.

22. Why should mean and standard deviation both be reported?

Mean represents the average performance, while standard deviation represents the variance between folds. A model with slightly higher mean but higher variance might not be very reliable.

23. Is the highest validation accuracy always sufficient to select a model?

No. The mean performance, variance, computational costs, training stability, and independent test performance should be evaluated as well.

15 Additional Exercise

The manual requires two new combinations of learning rate, dropout, batch size and fine-tuning strategy to be evaluated using five-fold cross-validation.

The selected configuration used as the reference is:

RMSProp, $LR = 0.001$, $Batch = 32$, $Dropout = 0$, Frozen base.

For the final submission, two additional combinations should be evaluated using the same five-fold procedure. Their results should be inserted into the following table.

Configuration	LR	Batch	Dropout	Fine-Tuning	Mean $\pm$ SD
Selected	0.001	32	0	No	91.71 $\pm$ 0.74
Additional C4	—	—	—	—	—
Additional C5	—	—	—	—	—

The new configurations should be judged using validation accuracy, standard deviation, computational cost and, where applicable, final test performance.

16 Conclusion

This experiment offers a thorough investigation of the impact of several key factors on CNN training and transfer learning based on MobileNetV2 in the Oxford-IIIT Pet dataset. The influence of initialization method, regularization technique, Batch Normalization, different optimization methods and training hyperparameters is evaluated experimentally.

The Zero initialization gave the best validation accuracy among the tested initialization techniques. Among the compared optimizers, the RMSProp gave the best performance in terms of validation accuracy. Learning rate 0.001 and zero dropout rate produced the best results among the tested combinations of learning rate and dropout rate. The good compromise between the quality of the model and its training speed is reached with batch size equal to 32.

For the feature extraction technique, fine-tuning gave worse validation accuracy among the compared configurations. After five-fold cross-validation, the Zero-initialized configuration with RMSProp as an optimizer and learning rate 0.001 and batch size 32 with zero dropout was chosen, producing the average validation accuracy of 91.71% with standard deviation of 0.74%.

After the final training on all available training data, the model gave 89.97% test accuracy, 90.54% macro precision, 89.91% macro recall and 89.93% macro F1-score. The confusion matrix shows good performance on the diagonal for almost all breeds while similar visually breeds give relatively large share of mistakes.

In general, this experiment shows that model selection should not be based on only one validation accuracy.

17 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>